COLA DE IMPRESIÓN COMPARTIDA CON IPC

Sincronización productor-consumidor entre procesos con memoria compartida y semáforos POSIX

Gaitan Velandia Diego Alejandro - Pruebas Ejecución de escenarios y validación de resultados.
López Tovar Mariana - Documentación técnica, organización del informe y registro de evidencias.
Montealegre Pinzón Carlos Esteban – Programación, Implementación e integración de componentes
dalegaitan@poligran.edu.co, mlopezt@poligran.edu.co, cespinzon@poligran.edu.co
(Aunque se asignaron responsabilidades principales, los tres integrantes participaron conjuntamente en el desarrollo, revisión y construcción del proyecto)
Grupo No.7

Checkpoint: N° 4
Fecha de entrega: 04.11.2026

Resumen

Motivación.
En un sistema operativo, varios procesos independientes pueden compartir recursos como una impresora. Sin coordinación, pueden aparecer trabajos duplicados, datos sobrescritos o procesos bloqueados. El proyecto permite observar el problema productor-consumidor y comparar el comportamiento de una cola sincronizada con una variante experimental sin sincronización.

Objetivo.
Desarrollar en Java, sobre Kali Linux, una cola de impresión compartida en la que procesos cliente generan trabajos y procesos impresora los consumen, utilizando memoria mapeada y semáforos POSIX para coordinar el acceso a la cola.

Conceptos aplicados.
Comunicación entre procesos, memoria compartida mediante un archivo mapeado, sincronización productor-consumidor con semáforos POSIX con nombre, exclusión mutua, gestión de procesos y permisos POSIX.
Método y resultados.
La cola circular reside en /dev/shm y es compartida mediante un archivo mapeado. Los semáforos POSIX mutex, vacios y llenos coordinan el acceso y la disponibilidad de espacios y trabajos. FileLock se utiliza para serializar las escrituras de la bitácora. En la prueba sincronizada de 3 clientes, 2 impresoras y 4

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

trabajos por cliente se registraron 12 trabajos terminados. En la variante experimental sin sincronización se observaron identificadores repetidos, solo 9 finalizaciones registradas y activación del límite de 30 segundos.

Palabras clave

comunicación entre procesos, memoria compartida, semáforos, producto-consumidor

Alcance

El proyecto se desarrollará como una aplicación de línea de comandos en Java para Linux, con una arquitectura cliente-servidor local: todos los procesos se ejecutan en la misma máquina y se comunican exclusivamente mediante mecanismos que ofrece el sistema operativo (memoria compartida y bloqueos sobre archivos).

El proyecto si incluirá:

- Un proceso servidor que inicializa el archivo compartido en `/dev/shm`, crea los recursos de sincronización y lanza los procesos hijos.
- Procesos cliente (productores) que generan e insertan trabajos en la cola compartida.
- Procesos impresora (consumidores) que retiran trabajos, simulan su impresión y registran los eventos.
- Una cola circular de capacidad limitada implementada mediante `MappedByteBuffer`.
- Semáforos POSIX con nombre: `mutex`, `vacíos` y `llenos`, encargados de controlar la exclusión mutua y la disponibilidad de espacios y trabajos.
- Una bitácora de eventos protegida mediante `FileLock`.
- Una variante experimental sin sincronización para evidenciar las condiciones de carrera.
- Controles de seguridad mediante permisos POSIX 0600, validación de los archivos y restricción de ejecución con privilegios de root.
- Pruebas funcionales y automáticas para verificar la sincronización y las políticas de seguridad.

El proyecto no incluirá:

- El proyecto no incluye impresión física, comunicación entre equipos mediante red, interfaz gráfica ni persistencia de los trabajos después de reiniciar el sistema. Tampoco contempla soporte para Windows o dispositivos móviles. La aplicación está diseñada para ejecutarse en Kali Linux y requiere JDK y JNA para acceder a las funciones de semáforos POSIX

Línea de proyecto elegida

Línea 7. Aplicación cliente-servidor con IPC.

El grupo desarrollará un programa con varios procesos que se comunican mediante memoria compartida y un mecanismo de semáforos para resolver un problema real de sincronización: la administración de una cola de impresión compartida por varios clientes y varias impresoras, que corresponde al patrón productor-consumidor aplicado a un caso concreto.

Problema que resuelve.

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

Si varios procesos escriben y leen a la vez sobre una misma cola sin coordinación, pueden sobrescribirse trabajos, perderse o atenderse dos veces, o quedar bloqueados cuando la cola está llena o vacía. El proyecto muestra ese fallo y su solución.

Plataforma.
Kali Linux, Java y las clases de java.nio para el archivo de mapeado; Process-Builder crea procesos independientes y JNA permite invocar semáforos POSIX de Linux. FileLock se utiliza en la bitácora.

HASTA AQUÍ LLEGA LA ENTREGA 1 — CHECKPOINT SEMANA 8 (Idea y alcance del proyecto)

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

INTRODUCCIÓN

Las colas de impresión son un perfecto ejemplo de recurso compartido, esto debido a que diferentes programas pueden solicitar trabajo a la vez que uno o varios procesos encargados de imprimir los atienden. Al varios procesos acceder simultáneamente a una cola sin coordinación, pueden sobrescribirse posiciones, hacer duplicaciones e incluso perdida de trabajos. Estos riesgos se relacionan con las condiciones de carrera y la sincronización entre procesos, conceptos centrales de los sistemas operacionales.

Motivación y Alcance

El proyecto propone una aplicación en Java sobre Linux que represente el patrón productor-consumidor, cliente-servidor mediante procesos independientes. Como segundo checkpoint se elaboró una primera versión funcional. Un proceso coordinador inicializa la cola, crea procesos cliente y procesos impresora e intercambian trabajos a través de un archivo ubicado en /dev/shm y proyectado en memoria. El acceso concurrente a la cola se coordina mediante semáforos POSIX con nombre, mientras que las escrituras concurrentes de la bitácora se serializan mediante FileLock. La impresión es simulada y no se cuenta con interfaz gráfica o comunicación por red.

El alcance de este avance incluye la creación de procesos, el intercambio de trabajos mediante memoria mapeada, la sincronización mediante semáforos POSIX, el registro de eventos y los permisos restrictivos de archivos. La variante sin sincronización se conserva únicamente para comparación experimental.

MÉTODO

Se eligió Java porque permite crear procesos independientes con ProcessBuilder y trabajar con archivos y memoria mapeada mediante java.nio. Para utilizar semáforos POSIX con nombre desde las JVM independientes, se incorporó JNA como puente hacia las funciones de linux.

- La solución está pensada para su ejecución en máquina virtual Kali Linux.
- El archivo compartido se ubica en dev/shm, directorio temporal respaldado por memoria en sistemas Linux.
- MappedByteBuffer representa la region compartida
- File Chanel administra el archivo
- Los semáforos POSIX mutex, vacíos y llenos coordinan las operaciones de la cola; FileLock serializa las estructuras de la bitácora.

La cola es circular y limitada, adicionalmente su cabecera almacena capacidad, índices de lectura y escritura, cantidad de elementos, numero de trabajos previstos y contador de trabajos terminados. Cada posición reserva un espacio fijo para el identificador y la descripción del trabajo. Cuando la cola está llena, el productor espera y vuelve a comprobar, cuando está vacía, el consumidor espera o finaliza si ya se completaron todos los trabajos previstos.

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

Herramientas y Componentes

Componente	Responsabilidad
ColaImpresion.server	Inicializar el archivo compartido, crear procesos hijos y esperar finalización
ColaImpresion.cliente	Actúa como productor: genera trabajos y los inserta en la cola.
ColaImpresion.printer	Actúa como consumidor: retira trabajos, simula su impresión y registra eventos.
MappedByteBuffer / File Channel	Permiten mapear y acceder al archivo que representa la cola compartida.
FileLock	Serializa las escrituras concurrentes de la bitácora;
Bitacora_impresion.log	Registra inicialización, encolado, inicio y fin de trabajos.

Arquitectura del avance

Flujo de ejecución

El servidor inicializa el archivo compartido en `/dev/shm`, proyecta la región de memoria mediante `MappedByteBuffer` y crea los semáforos POSIX necesarios para la sincronización. Posteriormente, utiliza `ProcessBuilder` para iniciar los procesos clientes (productores) e impresoras (consumidores).

Los clientes generan trabajos y, mediante los semáforos `vacios` y `mutex`, controlan la disponibilidad de espacios y el acceso exclusivo a la cola circular. Las impresoras utilizan los semáforos `llenos` y `mutex` para retirar los trabajos disponibles de manera coordinada. Finalmente, los eventos de encolado, inicio y finalización se registran en `bitacora_impresion.log`, cuyas escrituras concurrentes se protegen mediante `FileLock`.

hilos dentro de una sola instancia.

Procedimiento de construcción y prueba

1. Comprobar instalación de JDK con java –version y javac –version
2. Compilar desde la carpeta src con javac ColaImpresion.java
3. Ejecutar una prueba corta con java ColaImpresion server 2 2 2.
4. Revisar la bitácora con cat bitacora_impresion.log y verificar que los cuatro trabajos previstos aparezcan encolados y finalizados.
5. Repetir la ejecución con una configuración mayor, por ejemplo, java ColaImpresion server 3 2 4, y comparar el número de trabajos previstos con los eventos registrados.

Criterios de validación

La validación se realiza mediante la ejecución en Kali Linux, la revisión de la bitácora y la comparación entre el número de trabajos esperados y los finalizados. En la prueba sincronizada con 3 clientes, 2 impresoras y 4 trabajos por cliente, la bitácora registró 12 finalizaciones y todos los procesos terminaron. En la variante sin sincronización, la prueba registró 9 finalizaciones de 12 trabajos esperados, identificadores repetidos y activación del límite de tiempo de 30 segundos. Los permisos reales de

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

/dev/shm/cola_impresion_ipc.dat y bitacora_impresion.log se verificaron con stat y ambos mostraron modo 600.

Evidencias Funcionalidad

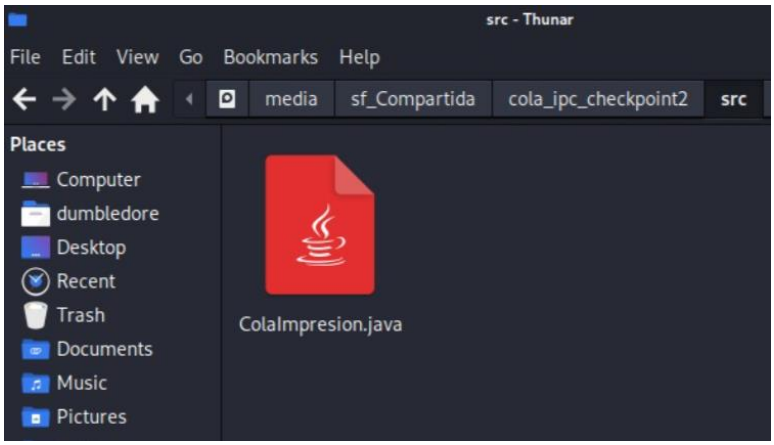


Imagen 1
Archivo ColaImpresion.java Creado (Primera Version Del Proyecto)

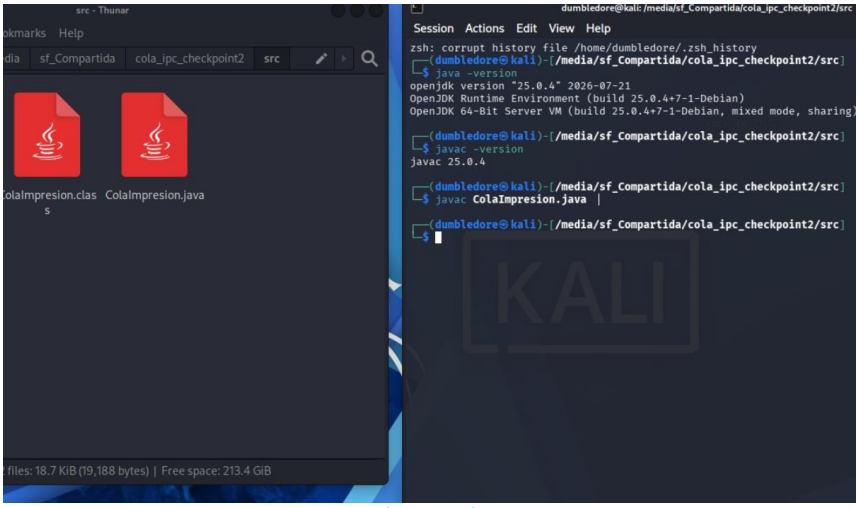


Imagen 2
Archivo Java y JavaC comprobacion y Compilacion (Primera Versión Del Proyecto)

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

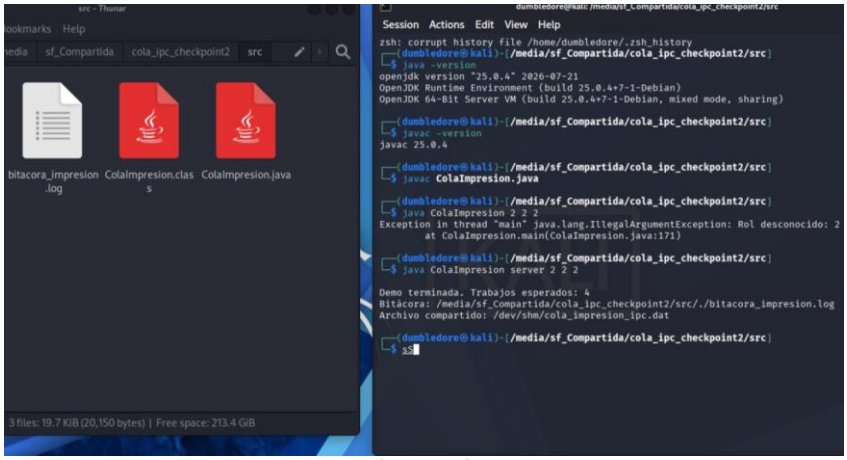


Imagen 3
Cola Impresion Compilado y Ejecutado (Primera Versión Del Proyecto)

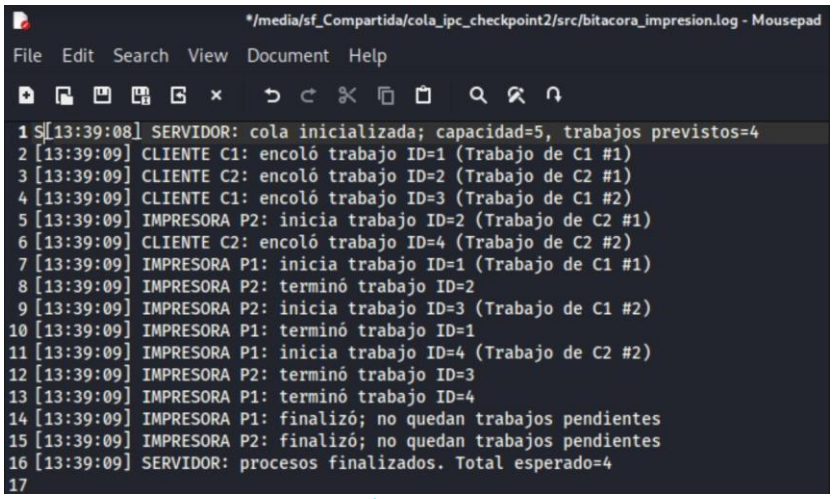


Imagen 4
Cola Impresión Compilado y Ejecutado (Primera Versión Del Proyecto)

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".



Imagen 5 (Segunda Versión Del Proyecto)

Repositorio GitHub

https://github.com/Evan-Roams/Colas-Impresion---Sistemas-Operacionales

Dificultades encontradas

Durante las pruebas se corrigieron problemas de configuración del entorno, permisos de ejecución del script y acceso a los archivos. La carpeta compartida de VirtualBox no aplicaba los permisos POSIX como se esperaba, por lo que las pruebas se trasladaron a una carpeta dentro del directorio personal de Kali Linux. También se corrigió una NonWritableChannelException en la variante experimental: el resumen final abría el archivo en solo lectura, aunque el método de mapeo solicitaba lectura y escritura. En la variante sin sincronización se observó una inconsistencia entre trabajos esperados y completados, junto con activación del límite de tiempo. Queda como limitación mejorar la recuperación ante terminaciones abruptas y automatizar pruebas repetidas.

HASTA AQUÍ LLEGA LA ENTREGA 2 — CHECKPOINT SEMANA 10 (Avance funcional)

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

Seguridad integrada

Resultados parciales

El proyecto cuenta con una versión funcional ejecutada en Kali Linux. En la prueba sincronizada se configuraron 3 procesos cliente, 2 procesos impresora y 4 trabajos por cliente, para un total esperado de 12. La bitácora registró 12 trabajos terminados y el servidor finalizó normalmente. También se comprobó que el archivo compartido /dev/shm/cola_impresion_ipc.dat y la bitácora bitacora_impresion.log tuvieran permisos POSIX 600. Como contraste, la variante experimental sin sincronización registró 9 finalizaciones para 12 trabajos esperados, mostró identificadores repetidos y activó el límite de 30 segundos. Estos resultados permiten evidenciar la importancia de coordinar el acceso concurrente. Se estima un avance funcional aproximado del 75 % del alcance: están implementados la cola compartida, los procesos productores y consumidores, los semáforos, el registro de eventos, los permisos y la prueba comparativa; falta fortalecer la recuperación ante caídas inesperadas y ampliar la batería de pruebas.

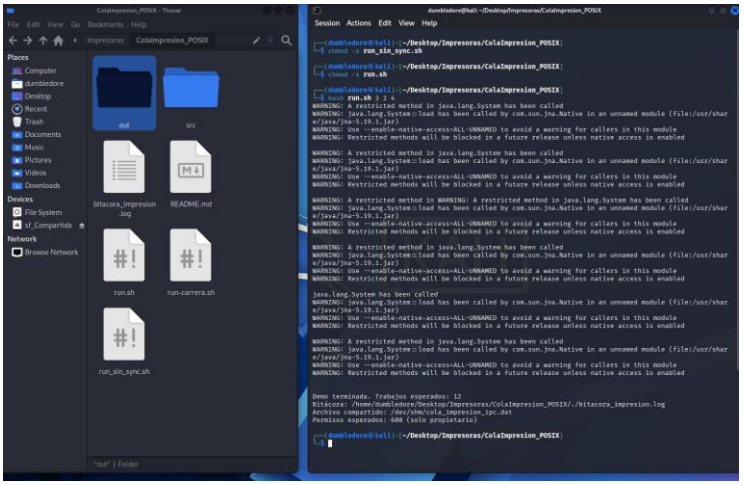


Imagen 6
Cola Impresión con semáforos POSIX (Segunda Versión Del Proyecto)

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

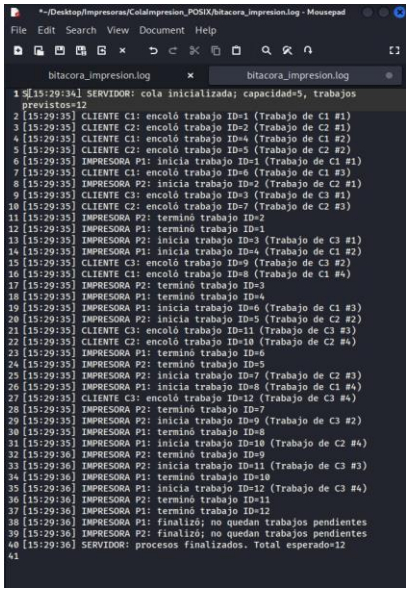


Imagen 7

Bitácora Impresión con semáforos POSIX (Segunda Versión Del Proyecto)

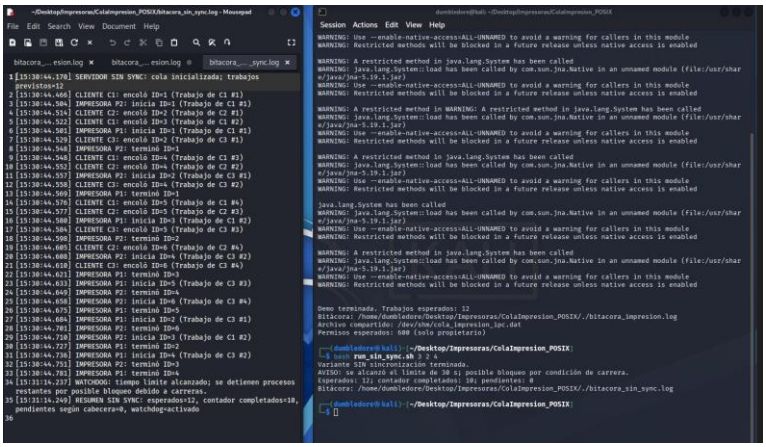


Imagen 8

Bitacora e Impresión con semáforos POSIX pero sin sincronización (Segunda Versión Del Proyecto)

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".



Imagen 9
Comprobación de permisos 600 solo Owner (Segunda Versión Del Proyecto)

Mecanismo de seguridad aplicado

El proyecto implementa varios controles de seguridad para proteger los recursos compartidos y limitar la ejecución de la aplicación. El primer mecanismo consiste en establecer permisos POSIX 0600 sobre el archivo de la cola compartida y la bitácora, permitiendo únicamente al propietario realizar operaciones de lectura y escritura.

Como segundo mecanismo, se incorporó una política de validación de seguridad que comprueba las condiciones del archivo compartido antes de utilizarlo. La aplicación verifica que el recurso sea un archivo regular, que no corresponda a un enlace simbólico y que sus permisos sean exactamente 0600. Si alguna condición no se cumple, el acceso es rechazado.

Adicionalmente, se implementó una restricción que impide ejecutar el servidor con privilegios de root. Esta política verifica el identificador de usuario del proceso y genera una excepción de seguridad cuando detecta que la aplicación está siendo ejecutada con UID 0.

Para comprobar estos mecanismos se desarrolló una prueba automática mediante SecurityPolicyTest.java y el script test-security.sh. La prueba verificó cuatro condiciones: aceptación de permisos 600, rechazo de permisos 664, aceptación del archivo después de restaurar los permisos 600 y comprobación de que el proceso de prueba no se ejecutara como root. El resultado obtenido fue de 4/4 comprobaciones superadas. Finalmente, se realizó una prueba independiente utilizando sudo, en la cual la aplicación rechazó la ejecución mediante una SecurityException con el mensaje de ejecución como root no permitida.

Justificación técnica

Los mecanismos implementados se relacionan con el control de acceso discrecional (DAC) y el principio de mínimo privilegio, conceptos abordados en los contenidos de seguridad de Sistemas Operacionales.

El modelo DAC permite establecer permisos sobre los recursos del sistema de archivos para determinar qué usuarios pueden leer o modificar un archivo. En este proyecto, el modo POSIX 0600 limita el acceso al propietario, evitando que otros usuarios del sistema puedan consultar o modificar directamente la cola compartida y la bitácora.

La validación de los permisos permite comprobar que el archivo cumple la política definida antes de utilizarlo, mientras que la restricción de ejecución con UID 0 evita que la aplicación utilice privilegios administrativos innecesarios. Esto se relaciona con el principio de mínimo privilegio, según el cual un proceso debe operar únicamente con los permisos necesarios para cumplir su función.

Es importante distinguir que los permisos y las restricciones de ejecución protegen el acceso a los recursos, mientras que los semáforos POSIX solucionan el problema de concurrencia entre procesos. Son mecanismos complementarios, pero cumplen funciones diferentes.

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

Estos controles tienen un alcance limitado: no proporcionan cifrado ni garantizan la integridad criptográfica de la bitácora, y la validación previa de permisos no elimina completamente los posibles cambios del archivo entre su comprobación y su apertura.

HASTA AQUÍ LLEGA LA ENTREGA 3 — CHECKPOINT SEMANA 13 (Integración con seguridad)

RESULTADOS

Funcionamiento de la cola compartida

La prueba principal se ejecutó en Kali Linux utilizando tres procesos cliente, dos procesos impresora y cuatro trabajos por cliente, para un total de 12 trabajos esperados. La ejecución se realizó mediante el comando ./run.sh 3 2 4.
Durante la prueba, los procesos productores insertaron los trabajos en la cola compartida y los procesos consumidores los retiraron para simular su impresión. La aplicación finalizó correctamente y mostró el mensaje de finalización con los 12 trabajos esperados. La bitácora permitió observar los eventos de inicialización, encolado, inicio y finalización de los trabajos.
Al comparar las variantes de sincronización, se observó una diferencia importante. La versión sincronizada completó los 12 trabajos, mientras que la variante sin sincronización registró únicamente 9 finalizaciones de los 12 trabajos esperados y activó el límite de tiempo de 30 segundos. También se observaron identificadores repetidos, lo que evidencia las inconsistencias que pueden producirse cuando varios procesos modifican un recurso compartido sin protección.

Validación de los controles de seguridad

Se verificaron los permisos del archivo /dev/shm/cola_impresion_ipc.dat y de bitacora_impresion.log mediante el comando stat. En ambos casos se obtuvo el modo 600 (-rw-------), confirmando que únicamente el propietario dispone de permisos de lectura y escritura.
Posteriormente, se ejecutó el script ./test-security.sh. La prueba automática obtuvo cuatro comprobaciones satisfactorias: aceptación de permisos 600, rechazo de permisos 664, aceptación del archivo después de restaurar los permisos 600 y verificación de que el usuario de prueba no fuera root.
También se realizó una prueba independiente ejecutando la aplicación mediante sudo. El sistema rechazó la operación mediante una excepción de seguridad, evitando que el servidor iniciara con privilegios administrativos.

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

En conjunto, los resultados muestran que la aplicación cumple su propósito de demostrar la comunicación entre procesos, la sincronización de una cola compartida y la aplicación de controles básicos de seguridad en Linux.

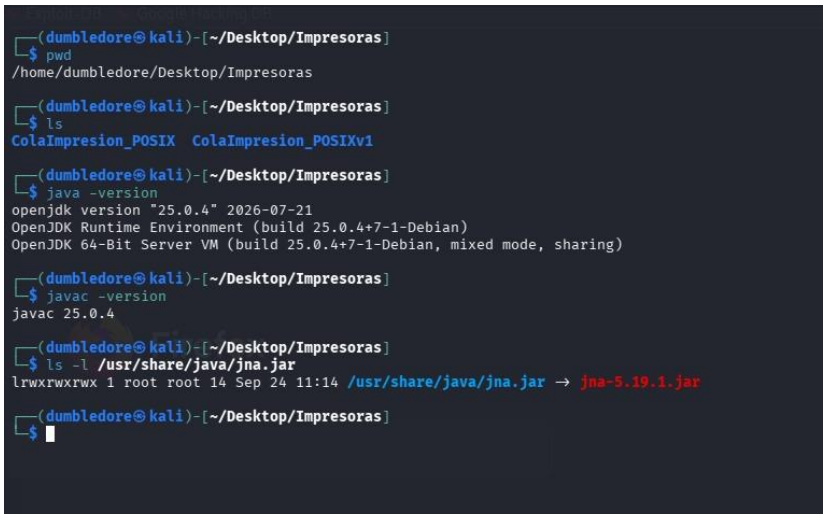


Imagen 10

Configuración del entorno de desarrollo en Kali Linux.

Nota. Se muestra la versión de OpenJDK instalada, la disponibilidad del compilador javac y la ubicación de la biblioteca JNA utilizada para integrar los semáforos POSIX desde Java.



"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

Imagen 11

Ejecución de pruebas automáticas de seguridad y funcionamiento de la aplicación.

Nota. Se observa el resultado de 4/4 comprobaciones satisfactorias del script de seguridad y la ejecución de la cola de impresión con una configuración de tres clientes, dos impresoras y cuatro trabajos por cliente. Para un total de 12 trabajos superados.

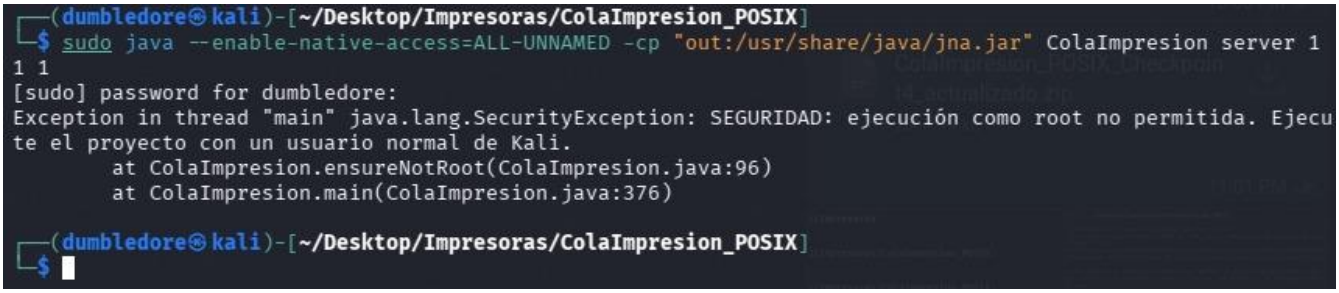


Imagen 12

Rechazo de ejecucion de la aplicacion con privilegios de root..

Nota. Al ejecutar el servidor mediante sudo, la aplicacion genera una SecurityException e impide continuar con la ejecucion, aplicando la politica de seguridad establecida.

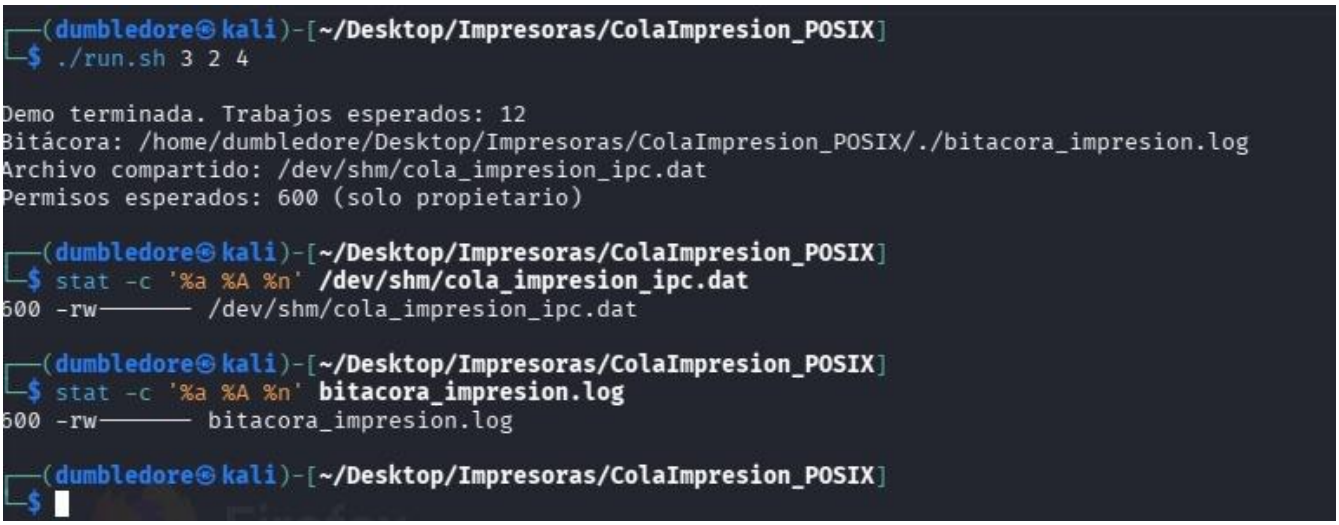


Imagen 13

Verificacion de permisos POSIX del archivo compartido y bitacora..

Nota. Usando el comando stat se comprueba que el archivo de la cola compartida y el de el registro tienen permisos 600, restringiendo la lectura y escritura solo al propietario

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".



Imagen 14

Resultado individual de las pruebas automáticas del mecanismo de seguridad.

Nota. La ejecucion de test-security.sh verifica la aceptación de permisos 600, el rechazo de permisos 664, la restauración de permisos correctos y la comprobación de que el proceso no se ejecute como root. Se obtuvieron cuatro comprobaciones satisfactorias.

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

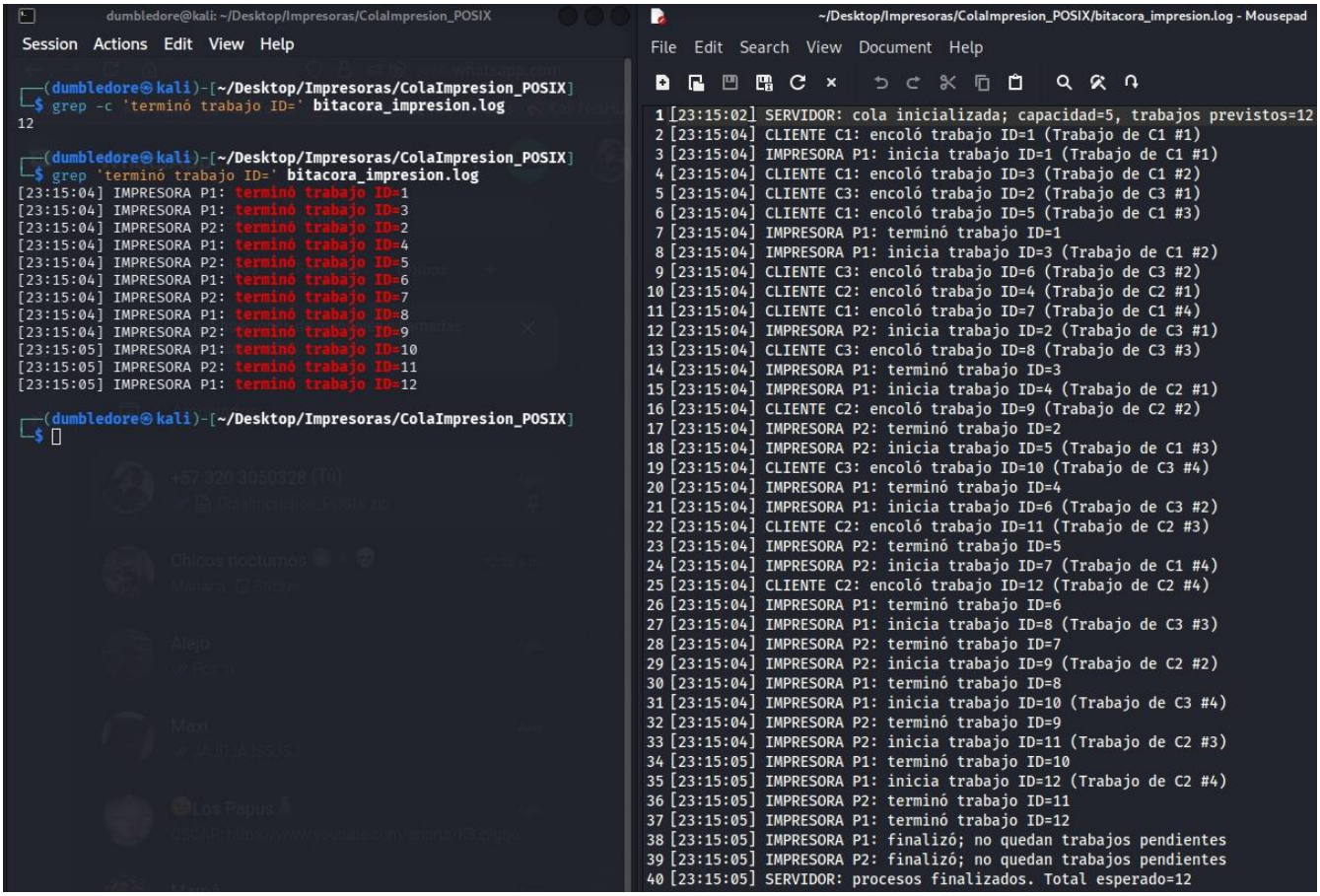


Imagen 15
Verificacion de trabajos finalizados mediante bitácora.

Nota. Se utiliza el comando grep para contar los registros de finalización almacenados en bitacora_impresion.log. El resultado obtenido fue de 12 trabajos terminados, coincidiendo con los 12 trabajos esperados en la prueba de ejecución.

DISCUSIÓN

El proyecto resolvió con éxito la sincronización en la cola de impresión compartida. Al implementar procesos productores y consumidores independientes, se logró simular un escenario real de acceso simultáneo a un recurso.

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

La comparativa entre ambas versiones deja clara la importancia de la exclusión mutua: sin sincronización se generan pérdidas de datos, IDs duplicados y tareas incompletas. Con los semáforos POSIX, en cambio, se regulan los espacios y trabajos disponibles, mientras que el semáforo *mutex* aísla la región crítica. Gracias a esto, la prueba completó los 12 trabajos programados sin errores.
En cuanto al control de acceso y seguridad, los permisos POSIX `0600`, la validación de archivos y la restricción para no ejecutar el sistema como *root* refuerzan la sincronización. Las pruebas automáticas y los ensayos con `sudo` confirmaron que las políticas responden bien en el entorno de pruebas.
Pese a cumplir los objetivos, el sistema tiene margen de mejora. Al usar un archivo temporal en `/dev/shm`, las tareas se pierden al reiniciar. Tampoco se incluyó un mecanismo de recuperación si un proceso se interrumpe a mitad de ejecución con un semáforo retenido, y la aplicación solo soporta una instancia activa del servidor a la vez.
En conclusión, si bien la solución alcanza los objetivos académicos y valida los conceptos de IPC, sincronización y seguridad, requeriría estos ajustes para dar el salto a un entorno de producción real.

CONCLUSIONES

Al desarrollar este proyecto se logro aplicar de manera práctica conceptos de comunicación entre procesos, memoria compartida, semáforos POSIX. La implementación de una cola de impresión compartida hizo posible comprender como es que diferentes procesos pueden colaborar mediante un recurso en común y el por qué es necesario controlar sus operaciones.
Al realizar la comparación entre la versión sincronizada y la variante sin sincronización, se evidencio las consecuencias que tiene las condiciones de "carrera". Mientras que la versión protegida completó los 12 trabajos previstos, la variante experimental presentó perdidas e inconsistencias. Cosa que permitió comprobar la importancia de utilizar mecanismos de sincronización adecuados en sistemas que comparten información.
También se incorporaron controles de seguridad relacionados con el control de acceso discrecional y el principio de mínimo privilegio. Los permisos POSIX 0600, la validación de los archivos y el rechazo de ejecución como root permitieron fortalecer la protección de los recursos utilizados por la aplicación.
Para futuras mejoras se planea implementar diversos mecanismos de recuperación antes fallos inesperados, sumado a una mejor administración de la creación y liberación de semáforos. Permitiendo la ejecución segura de varias instancias y así mismo ampliar las pruebas de estrés con diferentes cantidades de clientes e impresoras.
Finalmente el proyecto permitio relacionar conceptos y contenidos teóricos de Sistemas Operativos con una implementación que sea funcional, demostrando la importancia en cuanto a la sincronización y seguridad de recursos compartidos.

REFERENCIAS BIBLIOGRÁFICAS

- Linux man-pages project. (s. f.). *sem_overview(7) — Overview of POSIX semaphores*. https://man7.org/linux/man-pages/man7/sem_overview.7.html

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".

- Oracle. (2025). *Java Platform, Standard Edition 25 API Specification*. https://docs.oracle.com/en/java/javase/25/docs/api/
- Oracle. (2025). *ProcessBuilder (Java SE 25)*. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ProcessBuilder.html
- Oracle. (2025). *Files (Java SE 25)*. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/nio/file/Files.html
- Java Native Access contributors. (s. f.). *Java Native Access (JNA)*. GitHub. https://github.com/java-native-access/jna

(Liste aquí las fuentes consultadas para desarrollar su proyecto, en formato APA, última versión)

FIN DEL DOCUMENTO — ENTREGA 4: INFORME FINAL, CHECKPOINT SEMANA 16 (junto con la sustentación oral en clase)

"Este documento es propiedad intelectual del POLITECNICO GRANCOLOMBIANO, se prohíbe su reproducción total o parcial sin la autorización escrita de la Rectoría. TODO DOCUMENTO IMPRESO O DESCARGADO DEL SISTEMA, ES CONSIDERADO COPIA NO CONTROLADA".